SongSeeker: Large-Scale Song Search Engine By Lyrics

Jiachen Liang, Jiachen Liu, Joey Cai, Tong Tong
liang88@illinois.edu,jl315@illinois.edu,qcai20@illinois.edu,tong37@illinois.edu
University of Illinois Urbana-Champaign
Urbana, IL, USA

1 SUMMARY

SongSeeker is a large-scale lyrics-based song search engine. It allows users to describe songs naturally using mood, style preferences, lyric fragments, or artist names, and then returns the most relevant matches. By combining natural language understanding with large-scale indexing of lyrics, SongSeeker makes music discovery more intuitive, helping users find songs that match their emotions, themes, or favorite performers.

2 INTRODUCTION

Perhaps we have all experienced this: a melody echoes in our minds, or lyrics express our feelings, yet we just cannot recall the song title. Current traditional lyric search engines rely mainly on exact keyword matching, which requires users to input precise terms to find results, often inefficient or even ineffective. This limitation creates a significant 'semantic gap' problem: a vast disparity between the 'meaning' in the minds of users and the 'literal text' search engines can comprehend.

To bridge this gap, this project aims to design and implement a semantic-based lyrics search engine. Unlike traditional approaches, our system will transcend keyword constraints by focusing on understanding the intent and emotion behind user queries. Users will be able to describe a scene, a feeling, or a theme using natural language and receive semantically relevant results. This project primarily applies NLP and IR techniques based on the BM25 algorithm, specifically sentence embedding and vector retrieval, to solve a creative text retrieval problem.

3 DESCRIPTION OF INTENDED WORK

3.1 Data Pre-processing

During the data pre-processing phase, our goal is to clean and standardize the lyrics data for consistent retrieval later. First, we unify the schema to ensure each record has fixed fields such as song_id, title, artist, lyrics and year, while removing missing or duplicate entries. Next, we perform words normalization. Subsequently, we remove special characters, cleaning out HTML tags, punctuation, emojis, and irrelevant symbols (e.g., ♪, —, …) to retain only plain text. The final step involves removing non-lyrical content, eliminating sections unrelated to lyrics—such as structural tags like [Chorus], [Verse 1], and copyright information—ensuring retrieval is based solely on the lyric body.

3.2 Query Pre-processing

To ensure user queries align with the lyric data, we apply identical pre-processing steps to queries. This includes words normalization, removing special characters, and optionally performing word weighting—assigning higher weights to specific keywords. For example, when a user enters "love!!!", we can increase the weight of "love" to enhance matching accuracy. This step can be temporarily omitted if time is limited.

3.3 Keyword-matching Retrieval (TF-IDF / BM25)

For keyword-matching retrieval, we build an inverted index and use TF-IDF or BM25 algorithms to calculate similarity between queries and lyrics. This returns the top k most relevant lyrics or songs. The advantage of keyword retrieval is its precision, making it ideal for queries containing explicit terms like "love" or "heart." However, it struggles to understand semantic relationships, making it difficult to match synonyms like 'sad' and "heartbroken."

3.4 Semantic Retrieval (Embedding-based)

To address keyword retrieval's limitations, we incorporate a semantic retrieval module. This involves using pre-trained models (e.g., Sentence-BERT (SBERT)) to convert lyrics and queries into vector representations, then calculating similarity through methods like cosine similarity or vector indexing techniques such as FAISS. Semantic retrieval captures underlying word meanings—e.g., "lonely nights" and "feeling alone" are recognized as similar. However, it may introduce noise, necessitating comparison with keyword-matching retrieval results.

3.5 Result Comparison and Post-processing

During result processing, we compare and optimize outcomes from both retrieval methods. First, we employ a hybrid search approach combining keyword-matching retrieval with semantic retrieval. For instance, the final score could be set as $0.5 \times \text{BM25} + 0.5 \times$embedding similarity, with weights adjusted through experimentation. Subsequently, post-processing is performed, including removing duplicate segments and optimizing sorting to prioritize complete song results over fragmented lyrics lines. Finally, evaluation is conducted. Quantitatively, we manually annotate the relevance of query results to compute Precision@k, Recall@k, and NDCG@k. Qualitatively, we use the system to assess whether the retrieval results align with intuition and expectations.

3.6 Optional: Recommendation System

If time permits, we will extend the implementation to include a recommendation module. This module comprises song-based recommendation (recommending similar songs based on the song provided) and history-based recommendation (suggesting songs with similar themes based on user's search history). As this component requires additional engineering effort (e.g., we need to create a database for users' information and their search history), it is treated as an optional feature. The core objective remains ensuring the stability and effectiveness of the search engine itself.